Week 4 - Feature Engineering

Goal

Build a non-leaking feature for a gridded environmental dataset and compare model evaluation strategies.

In this activity, you will use a gridded **sea surface temperature (SST)** dataset from oceanography. SST varies smoothly across space and seasonally through time, so nearby observations are not independent.

Dataset: NOAA Extended Reconstructed Sea Surface Temperature version 5. NOAA describes ERSSTv5 as a global monthly SST analysis on a 2-degree grid derived from ICOADS, with records extending from 1854 to the present.

Useful dataset references:

- https://www.psl.noaa.gov/data/gridded/data.noaa.ersst.v5.html
- https://downloads.psl.noaa.gov/Datasets/noaa.ersst.v5/sst.mnmean.nc

1. Setup

Run the chunk below. If any package is missing, uncomment the install line and run it once.

This R Markdown document intentionally gives you some complete code and some incomplete code. Sections marked **TODO** are places where you should write or modify code yourself.

```
# Uncomment if needed:
# install.packages(c("tidymodels", "terra", "lubridate", "ranger"))

library(tidymodels)
library(terra)
library(lubridate)

theme_set(theme_minimal())

set.seed(42)
```

2. Load the Sea Surface Temperature Dataset

The data are stored as a NetCDF file, which is common for gridded environmental and oceanographic data. Instead of rows and columns only, the data have dimensions such as `time`, `lat`, and `lon`.

The SST variable is measured over a regular latitude-longitude grid through time.

```
#sst_url <- "https://downloads.psl.noaa.gov/Datasets/noaa.ersst.v5/sst.mnmean.nc"
#sst_file <- "sst.mnmean.nc"

#download.file(sst_url, sst_file, mode = "wb")
sst_all <- terra::rast("sst.nc")
sst_all
```

```
## class       : SpatRaster
## size        : 31, 43, 116  (nrow, ncol, nlyr)
## resolution  : 8.52381, 5.866667  (x, y)
```

```
## extent      : -4.261905, 362.2619, -90.93333, 90.93333  (xmin, xmax, ymin, ymax)
## coord. ref. :
## source      : sst.nc
## names       : sst_1, sst_2, sst_3, sst_4, sst_5, sst_6,  ...
## unit        : degC
## time (days) : 1992-01-01 to 2020-10-01 (116 steps)
```

3. Subset the Data

The full global dataset is larger than we need for this activity. The chunk below selects a broad Pacific Ocean region and thins the grid and time axis so the model runs quickly.

We are still keeping enough data to preserve spatial and temporal correlation.

```
sst_dates <- time(sst_all)

keep_time <- which(
  sst_dates >= as.Date("1992-01-01") &
    sst_dates <= as.Date("2020-12-31")
)

# Keep every third month.
keep_time <- keep_time[seq(1, length(keep_time), by = 3)]

sst_sub <- sst_all[[keep_time]]

# Select a broad Pacific region.
# Longitude is stored from 0 to 360 degrees in this dataset.
sst_sub <- crop(sst_sub, ext(120, 290, -60, 60))

# Thin the grid by aggregating neighboring cells.
sst_sub <- aggregate(sst_sub, fact = 2, fun = mean, na.rm = TRUE)

dates_sub <- sst_dates[keep_time]
names(sst_sub) <- paste0("t", seq_along(dates_sub))

sst_sub
```

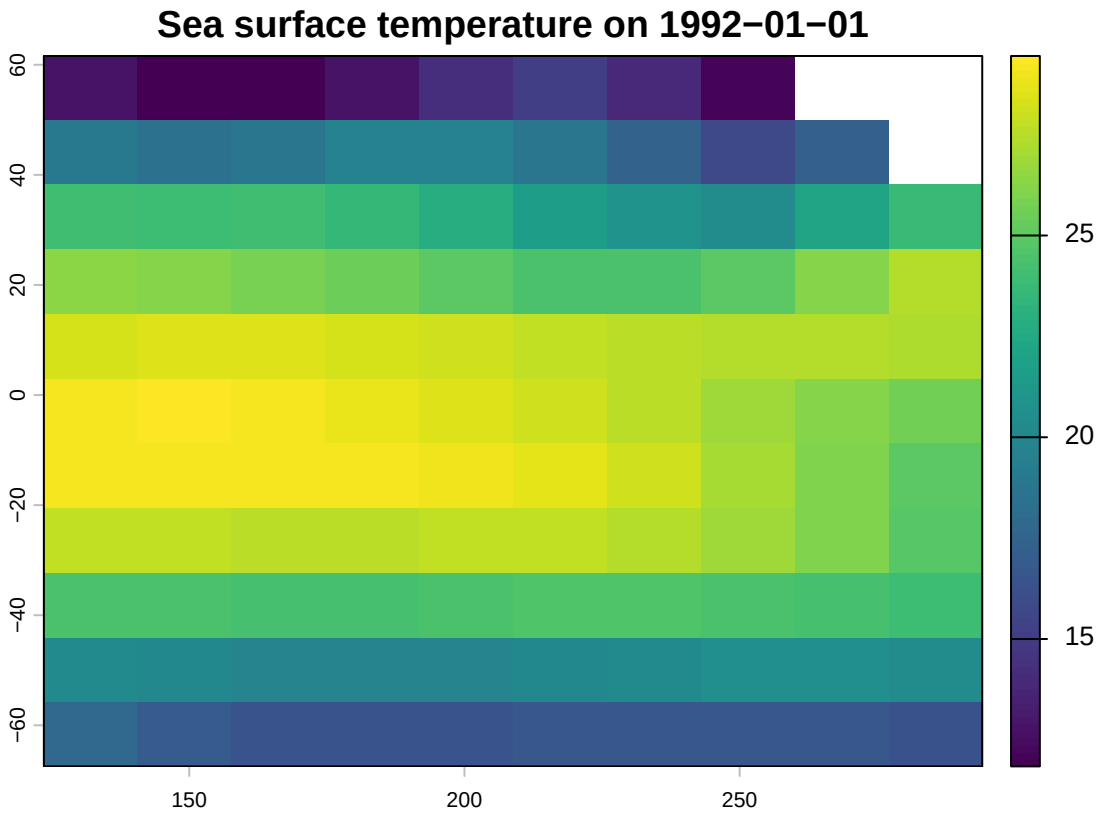
```
## class       : SpatRaster
## size        : 11, 10, 39  (nrow, ncol, nlyr)
## resolution  : 17.04762, 11.73333  (x, y)
## extent      : 123.5952, 294.0714, -67.46667, 61.6  (xmin, xmax, ymin, ymax)
## coord. ref. :
## source(s)   : memory
## names       :       t1,       t2,       t3,       t4,       t5,       t6, ...
## min values  : 11.84346, 11.96796, 12.04149, 11.70802, 12.41832, 12.19061, ...
## max values  : 29.44872, 29.64496, 29.58152, 29.76559, 29.66863, 29.10538, ...
```

4. Explore the Data Visually

Before modeling, inspect the target variable. SST has strong geographic structure and seasonal/temporal structure.

The first plot shows the SST field for one time step. Nearby ocean cells usually have similar temperatures, which is spatial autocorrelation.

```r
plot(sst_sub[[1]], main = paste("Sea surface temperature on", dates_sub[1]))
```

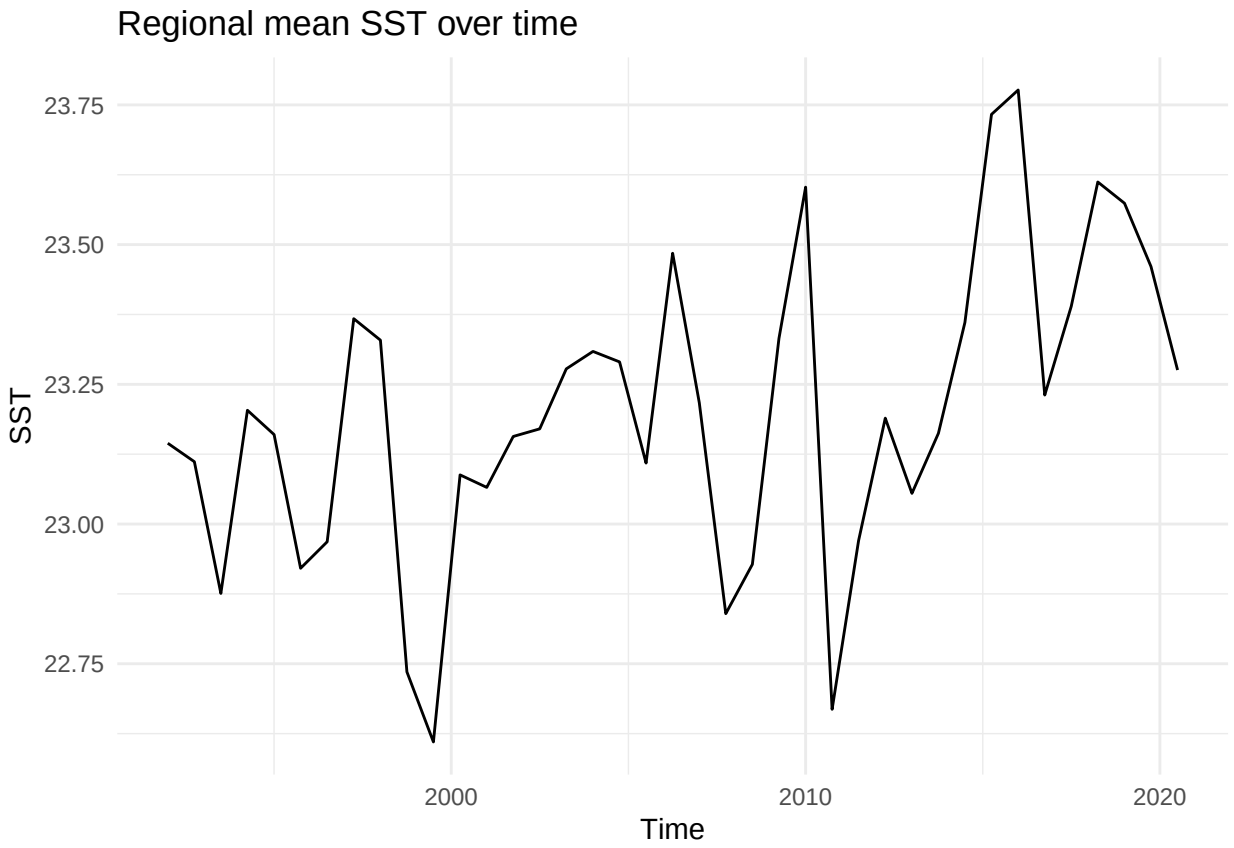


Explore Change Through Time

The next plot averages SST across the selected region for each time step. Because we kept every third month, the pattern will still show seasonal and long-term variation.

```r
regional_mean <- global(sst_sub, fun = mean, na.rm = TRUE) %>%
  as_tibble() %>%
  mutate(time = dates_sub)

ggplot(regional_mean, aes(x = time, y = mean)) +
  geom_line() +
  labs(
    title = "Regional mean SST over time",
    x = "Time",
    y = "SST"
  )
```



5. Convert Gridded Data to a Machine-Learning Table

Most machine-learning models expect a table where each row is one observation. Here, one row represents one grid cell at one time.

The target is `sst`. The predictors will be simple features based on location and time.

We create cyclic features for month because December is close to January.

```
df_wide <- as.data.frame(sst_sub, xy = TRUE, na.rm = TRUE)

date_lookup <- tibble(
  layer = names(sst_sub),
  time = dates_sub
)

df <- df_wide %>%
  pivot_longer(
    cols = starts_with("t"),
    names_to = "layer",
    values_to = "sst"
  ) %>%
  left_join(date_lookup, by = "layer") %>%
  rename(lon = x, lat = y) %>%
  mutate(
    year = year(time),
    month = month(time)
  )
```

```r
# TODO: create month_sin and month_cos features.
# Hint: use sin(2 * pi * month / 12) and cos(2 * pi * month / 12).

df <- df %>%
  mutate(
    month_sin = sin(2 * pi * month / 12), # TODO: replace with month sine feature
    month_cos = cos(2 * pi * month / 12) # TODO: replace with month cosine feature
  )

dim(df)
```

```
## [1] 4173    9
```

```r
head(df)
```

```
## # A tibble: 6 x 9
##     lon   lat layer   sst time        year month month_sin month_cos
##   <dbl> <dbl> <chr> <dbl> <date>     <dbl> <dbl>     <dbl>     <dbl>
## 1  132.  55.7 t1     12.8 1992-01-01  1992     1     0.5       0.866
## 2  132.  55.7 t2     18.9 1992-10-01  1992    10    -0.866     0.5
## 3  132.  55.7 t3     17.8 1993-07-01  1993     7    -0.5      -0.866
## 4  132.  55.7 t4     12.4 1994-04-01  1994     4     0.866    -0.5
## 5  132.  55.7 t5     13.2 1995-01-01  1995     1     0.5       0.866
## 6  132.  55.7 t6     18.8 1995-10-01  1995    10    -0.866     0.5
```

6. Define the Prediction Problem

We will predict SST from simple location and time features.

This is not meant to be the best possible ocean forecasting model. The goal is to study how evaluation changes when data are spatially and temporally correlated.

```r
feature_cols <- c(
  "lat",
  "lon",
  "month_sin",
  "month_cos",
  "year"
)

target_col <- "sst"

model_data <- df %>%
  select(all_of(feature_cols), all_of(target_col))

head(model_data)
```

```
## # A tibble: 6 x 6
##     lat   lon month_sin month_cos  year   sst
##   <dbl> <dbl>     <dbl>     <dbl> <dbl> <dbl>
## 1  55.7  132.     0.5       0.866  1992  12.8
## 2  55.7  132.    -0.866     0.5    1992  18.9
## 3  55.7  132.    -0.5      -0.866  1993  17.8
## 4  55.7  132.     0.866    -0.5    1994  12.4
## 5  55.7  132.     0.5       0.866  1995  13.2
## 6  55.7  132.    -0.866     0.5    1995  18.8
```

7. Create a Reusable Model and Evaluation Function

We use a tree-based regression model through `tidymodels`. The `ranger` engine fits a random forest. It can capture nonlinear patterns such as warmer temperatures near the equator and seasonal variation.

The helper function below runs resampling and returns RMSE, MAE, and traditional R-squared.

Interpretation of metrics:

- **RMSE:** Typical prediction error, with larger errors penalized more strongly.
- **MAE:** Typical absolute error.
- **R-squared:** Fraction of variance explained. Higher is better, but it can be misleading when validation folds are not independent.

```
regression_metrics <- metric_set(rmse, mae, rsq_trad)

make_workflow <- function(features) {
  formula_text <- paste("sst ~", paste(features, collapse = " + "))

  sst_recipe <- recipe(as.formula(formula_text), data = model_data) %>%
    step_impute_median(all_numeric_predictors())

  rf_spec <- rand_forest(
    trees = 200,
    min_n = 5
  ) %>%
    set_engine("ranger") %>%
    set_mode("regression")

  workflow() %>%
    add_recipe(sst_recipe) %>%
    add_model(rf_spec)
}

summarize_resamples <- function(results, label) {
  collect_metrics(results) %>%
    filter(.metric %in% c("rmse", "mae", "rsq_trad")) %>%
    mutate(Evaluation = label) %>%
    select(Evaluation, .metric, mean, std_err)
}
```

8. Naive Evaluation: Random k-Fold Cross-Validation

Run the chunk below and record the results.

Random k-fold cross-validation mixes nearby places and nearby times across folds. That can make performance look better than it would be when predicting genuinely new regions or future periods.

```
set.seed(42)

random_folds <- vfold_cv(model_data, v = 5)

random_results <- fit_resamples(
  make_workflow(feature_cols),
  resamples = random_folds,
  metrics = regression_metrics
)
```

```
random_summary <- summarize_resamples(random_results, "Random k-fold CV")
random_summary
```

```
## # A tibble: 3 x 4
##   Evaluation       .metric   mean  std_err
##   <chr>            <chr>    <dbl>    <dbl>
## 1 Random k-fold CV mae      0.714 0.00893
## 2 Random k-fold CV rmse     0.891 0.0102
## 3 Random k-fold CV rsq_trad 0.968 0.000853
```

9. Feature Engineering

TODO: Add a historic monthly average temperature for each location.

Be careful not to use future data when creating this column. For each grid cell and month, the value should be based only on previous observations from the same grid cell and calendar month.

```
#glimpse(df)

df <- df %>%
  arrange(lat, lon, month, time) %>%
  group_by(lat, lon, month) %>%
  mutate(
    Month_Average_sst = dplyr::lag(cummean(sst)) # TODO: replace with historical monthly average
  ) %>%
  ungroup() %>%
  drop_na()

df %>%
  select(lat, lon, time, month, sst, Month_Average_sst) %>%
  arrange(lat, lon, month, time) %>%
  head(15)
```

```
## # A tibble: 15 x 6
##      lat   lon time       month   sst Month_Average_sst
##    <dbl> <dbl> <date>     <dbl> <dbl>             <dbl>
##  1 -61.6  132. 1995-01-01     1  17.9              17.7
##  2 -61.6  132. 1998-01-01     1  18.0              17.8
##  3 -61.6  132. 2001-01-01     1  18.4              17.9
##  4 -61.6  132. 2004-01-01     1  18.7              18.0
##  5 -61.6  132. 2007-01-01     1  17.7              18.1
##  6 -61.6  132. 2010-01-01     1  17.7              18.1
##  7 -61.6  132. 2013-01-01     1  18.7              18.0
##  8 -61.6  132. 2016-01-01     1  17.8              18.1
##  9 -61.6  132. 2019-01-01     1  19.3              18.1
## 10 -61.6  132. 1997-04-01     4  17.6              16.8
## 11 -61.6  132. 2000-04-01     4  17.4              17.2
## 12 -61.6  132. 2003-04-01     4  18.0              17.3
## 13 -61.6  132. 2006-04-01     4  18.1              17.5
## 14 -61.6  132. 2009-04-01     4  17.4              17.6
## 15 -61.6  132. 2012-04-01     4  18.0              17.6
```

Hint: one safe pattern is to combine `cummean()` with `lag()` so the current row is not included in its own historical average.

```r
feature_cols <- c(
  "lat",
  "lon",
  "month_cos",
  "month_sin",
  "year",
  "Month_Average_sst"
)

model_data <- df %>%
  select(all_of(feature_cols), all_of(target_col))

head(model_data)
```

```
## # A tibble: 6 x 7
##     lat   lon month_cos month_sin  year Month_Average_sst   sst
##   <dbl> <dbl>     <dbl>     <dbl> <dbl>             <dbl> <dbl>
## 1 -61.6  132.     0.866       0.5  1995              17.7  17.9
## 2 -61.6  132.     0.866       0.5  1998              17.8  18.0
## 3 -61.6  132.     0.866       0.5  2001              17.9  18.4
## 4 -61.6  132.     0.866       0.5  2004              18.0  18.7
## 5 -61.6  132.     0.866       0.5  2007              18.1  17.7
## 6 -61.6  132.     0.866       0.5  2010              18.1  17.7
```

```r
set.seed(42)

random_folds <- vfold_cv(model_data, v = 5)

random_results_with_feature <- fit_resamples(
  make_workflow(feature_cols),
  resamples = random_folds,
  metrics = regression_metrics
)

random_summary_with_feature <- summarize_resamples(
  random_results_with_feature,
  "Random k-fold CV with historical monthly mean"
)

random_summary_with_feature
```

```
## # A tibble: 3 x 4
##   Evaluation                                      .metric   mean  std_err
##   <chr>                                           <chr>    <dbl>    <dbl>
## 1 Random k-fold CV with historical monthly mean mae      0.407 0.00995
## 2 Random k-fold CV with historical monthly mean rmse     0.536 0.0135
## 3 Random k-fold CV with historical monthly mean rsq_trad 0.988 0.000768
```